

LAPORAN PRATIKUM

STRUKTUR DATA

“Pratikum Pekan 6”

Disusun Oleh:

Rafikhul Ramadhan

2511533012

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Muhammad Galis Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, taufik, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk pemenuhan tugas akademik, sekaligus sebagai sarana untuk memperdalam pemahaman penulis terhadap konsep-konsep dasar dalam struktur data. Melalui praktikum ini, saya mempelajari berbagai materi penting seperti penggunaan array, stack, queue, linked list, serta bagaimana mengimplementasikannya dalam bahasa pemrograman. Selain itu, kegiatan praktikum juga melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Pada kesempatan ini, saya juga ingin menyampaikan rasa terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bimbingan, dukungan, dan bantuan selama proses praktikum hingga penyusunan laporan ini.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi penulis khususnya, serta bagi para pembaca pada umumnya. Penulis berharap laporan ini juga dapat digunakan sebagai referensi dalam memahami materi praktikum, baik pada mata kuliah Struktur Data maupun praktikum lainnya.

Padang, 14 Mei 2026

Rafikhul Ramadhan

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN.....	2
2.1 Langkah Kerja.....	2
2.2 NodeDLL_2511533012	2
2.3 InsertDLL_2511533012.....	3
2.4 PenelusuranDLL_2511533012	5
2.5 HapusDLL_2511533012	7
BAB III KESIMPULAN	10
3.1 Ringkasan.....	10
3.2 Kesimpulan.....	10
DAFTAR PUSTAKA	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum pekan ke-6, mahasiswa mempelajari konsep **Doubly Linked List (DLL)** dalam bahasa pemrograman Java. Doubly Linked List merupakan pengembangan dari Single Linked List, di mana setiap node memiliki dua pointer, yaitu pointer ke node berikutnya dan pointer ke node sebelumnya. Dengan adanya dua arah hubungan tersebut, proses penelusuran data dapat dilakukan dari depan maupun belakang.

Praktikum ini bertujuan untuk memberikan pemahaman mengenai struktur node pada Doubly Linked List serta operasi-operasi dasar seperti penambahan, penghapusan, dan penelusuran data.

1.2 Tujuan

Tujuan dari praktikum pekan ke-6 adalah untuk memahami konsep dasar Doubly Linked List, mengetahui hubungan antar node menggunakan pointer next dan prev, serta mampu mengimplementasikan operasi dasar Doubly Linked List dalam bahasa Java.

1.3 Manfaat

Manfaat dari praktikum ini adalah mahasiswa dapat memahami pengelolaan data dua arah menggunakan Doubly Linked List, meningkatkan kemampuan logika pemrograman, serta mampu menerapkan operasi penambahan, penghapusan, dan traversal data pada struktur data linked list.

BAB II

PEMBAHASAN

2.1 Langkah Kerja

Tahapan yang dilakukan dalam pratikum ini adalah sebagai berikut:

1. Menjalankan aplikasi IDE (seperti Eclipse atau IntelliJ IDEA) atau menggunakan text editor yang mendukung bahasa Java.
2. Membuat sebuah project baru dengan nama **pekan6_2511533012**.
3. Menambahkan tiga buah file Java, yaitu NodeDLL_2511533012.java, InsertDLL_2511533012.java, PenelusuranDLL_2511533012.java, HapusDLL_2511533012.java.
4. Menuliskan kode program sesuai dengan instruksi yang diberikan pada masing-masing file.
5. Melakukan proses kompilasi menggunakan *Java compiler* agar program dapat dijalankan.
6. Menjalankan program untuk mengamati hasil keluaran sesuai dengan logika yang dibuat.
7. Mengevaluasi hasil eksekusi pada *console* serta menarik kesimpulan dari setiap percobaan yang dilakukan.

2.2 NodeDLL_2511533012

```
package pekan6_2511533012;

public class NodeDLL_2511533012 {
    int data_3012;
    NodeDLL_2511533012 next_3012;
    NodeDLL_2511533012 prev_3012;

    public NodeDLL_2511533012(int data_3012) {
        this.data_3012 = data_3012;
        this.next_3012 = null;
        this.prev_3012 = null;
    }
}
```

Analisis:

Program ini merupakan class dasar untuk membentuk **node** pada **Doubly Linked List**.

Class memiliki atribut data, next, dan prev. Variabel data digunakan untuk menyimpan nilai, sedangkan next dan prev digunakan untuk menghubungkan node berikutnya dan sebelumnya.

Kesimpulan: Program ini menjadi dasar pembentukan struktur Doubly Linked List.

2.3 InsertDLL_2511533012

```
package pekan6_2511533012;

public class InsertDLL_2511533012 {
    static NodeDLL_2511533012 insertBegin_3012(NodeDLL_2511533012
head, int data) {
        NodeDLL_2511533012 new_node = new
NodeDLL_2511533012(data);

        new_node.next_3012 = head;

        if (head != null) {
            head.prev_3012 = new_node;
        }
        return new_node;
    }
    public static NodeDLL_2511533012 insertEnd_3012(NodeDLL_2511533012
head, int newData) {
        NodeDLL_2511533012 newNode = new NodeDLL_2511533012(newData);

        if (head == null) {
            head = newNode;
        } else {
            NodeDLL_2511533012 curr = head;
            while (curr.next_3012 != null) {
                curr = curr.next_3012;
            }
            curr.next_3012 = newNode;
            newNode.prev_3012 = curr;
        }
        return head;
    }
    public static NodeDLL_2511533012
insertAtPosition_3012(NodeDLL_2511533012 head, int pos, int
new_data) {
```

```

        NodeDLL_2511533012 new_node = new
NodeDLL_2511533012(new_data);
        if(pos == 1) {
            new_node.next_3012 = head;
            if (head != null) {
                head.prev_3012 = new_node;
            }
            head = new_node;
            return head;
        }
        NodeDLL_2511533012 curr = head;
        for (int i = 1; i < pos - 1 && curr != null; ++i) {
            curr = curr.next_3012;
        }
        if (curr == null) {
            System.out.println("Posisi tidak ada");
            return head;
        }
        new_node.prev_3012 = curr;
        new_node.next_3012 = curr.next_3012;
        curr.next_3012 = new_node;
        if (new_node.next_3012 != null) {
            new_node.next_3012.prev_3012 = new_node;
        }
        return head;
    }

    public static void printList_3012(NodeDLL_2511533012 head) {
        NodeDLL_2511533012 curr = head;
        while (curr != null) {
            System.out.print(curr.data_3012 + " <-> ");
            curr = curr.next_3012;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511533012 head = new NodeDLL_2511533012(2);
        head.next_3012 = new NodeDLL_2511533012(3);
        head.next_3012.prev_3012 = head;
        head.next_3012.next_3012 = new NodeDLL_2511533012(5);
        head.next_3012.next_3012.prev_3012 = head.next_3012;

        System.out.print("DLL Awal: ");
        printList_3012(head);

        head = insertBegin_3012(head, 1);
        System.out.print("simpul 1 ditambah di awal: ");
        printList_3012(head);

        System.out.print("simpul 6 ditambah di akhir: ");
        int data = 6;
        head = insertEnd_3012(head, data);
        printList_3012(head);
    }

```

```

        System.out.print("tambah node 4 di posisi 4: ");
        int data2 = 4;
        int pos = 4;
        head = insertAtPosition_3012(head, pos, data2);
        printList_3012(head);
    }
}

```

Output (2.2. NodeDLL_2511533012.java) :

```

<terminated> InsertDLL_2511533012 [Java Application] C:\Program Files\Java\jdk-24\bin\java
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

Analisis :

Program ini digunakan untuk melakukan operasi **penambahan node** pada Doubly Linked List.

Program memiliki beberapa method seperti:

- insertBegin untuk menambah node di awal
- insertEnd untuk menambah node di akhir
- insertAtPosition untuk menambah node pada posisi tertentu

Setelah proses penambahan, isi linked list ditampilkan menggunakan method printList.

Kesimpulan: Program ini menunjukkan berbagai cara penambahan node pada Doubly Linked List.

2.4 PenelusuranDLL_2511533012

```

package pekan6_2511533012;

public class PenelusuranDLL_2511533012 {
    static void forwardTraversal_3012(NodeDLL_2511533012 head) {
        NodeDLL_2511533012 curr = head;
    }
}

```

```

        while (curr != null) {
            System.out.print(curr.data_3012 + " <-> ");
            curr = curr.next_3012;
        }
        System.out.println();
    }

    static void backwardTraversal_3012(NodeDLL_2511533012 tail) {
        NodeDLL_2511533012 curr = tail;

        while (curr != null) {
            System.out.print(curr.data_3012 + " <-> ");
            curr = curr.prev_3012;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeDLL_2511533012 head = new NodeDLL_2511533012(1);
        NodeDLL_2511533012 second = new NodeDLL_2511533012(2);
        NodeDLL_2511533012 third = new NodeDLL_2511533012(3);

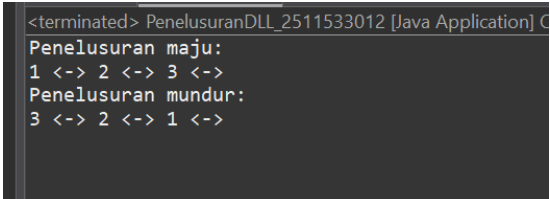
        head.next_3012 = second;
        second.prev_3012 = head;
        second.next_3012 = third;
        third.prev_3012 = second;

        System.out.println("Penelusuran maju: ");
        forwardTraversal_3012(head);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal_3012(third);
    }
}

```

Output :



```

<terminated> PenelusuranDLL_2511533012 [Java Application] C
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

Analisis :

Program ini digunakan untuk melakukan **penelusuran data** pada **Doubly Linked List**.

Program menyediakan dua method traversal, yaitu:

- forwardTraversal untuk penelusuran dari depan ke belakang
- backwardTraversal untuk penelusuran dari belakang ke depan

Hal ini dimungkinkan karena setiap node memiliki pointer next dan prev.

Kesimpulan: Program ini menunjukkan proses traversal dua arah pada Doubly Linked List.

2.5 HapusDLL_2511533012

```
package pekan6_2511533012;

public class HapusDLL_2511533012 {
    public static NodeDLL_2511533012
delHead_3012(NodeDLL_2511533012 head) {
        if (head == null) {
            return null;
        }
        NodeDLL_2511533012 temp = head;
        head = head.next_3012;
        if (head != null) {
            head.prev_3012 = null;
        }
        return head;
    }

    public static NodeDLL_2511533012
delLast_3012(NodeDLL_2511533012 head) {
        if (head == null) {
            return null;
        }
        if (head.next_3012 == null) {
            return null;
        }
        NodeDLL_2511533012 curr = head;
        while (curr.next_3012 != null) {
            curr = curr.next_3012;
        }

        if (curr.prev_3012 != null) {
            curr.prev_3012.next_3012 = null;
        }
        return head;
    }

    public static NodeDLL_2511533012
delPos_3012(NodeDLL_2511533012 head, int pos) {
        if (head == null) {
            return head;
        }
        NodeDLL_2511533012 curr = head;
```

```

        for (int i = 1; curr != null && i < pos; ++i) {
            curr = curr.next_3012;
        }
        if (curr == null) {
            return head;
        }
        if (curr.prev_3012 != null) {
            curr.prev_3012.next_3012 = curr.next_3012;
        }
        if (curr.next_3012 != null) {
            curr.next_3012.prev_3012 = curr.prev_3012;
        }
        if (head == curr) {
            head = curr.next_3012;
        }
        return head;
    }

    public static void printList_3012(NodeDLL_2511533012 head) {
        NodeDLL_2511533012 curr = head;
        while (curr != null) {
            System.out.print(curr.data_3012 + " ");
            curr = curr.next_3012;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeDLL_2511533012 head = new NodeDLL_2511533012(1);
        head.next_3012 = new NodeDLL_2511533012(2);
        head.next_3012.prev_3012 = head;
        head.next_3012.next_3012 = new NodeDLL_2511533012(3);
        head.next_3012.next_3012.prev_3012 = head.next_3012;
        head.next_3012.next_3012.next_3012 = new
NodeDLL_2511533012(4);
        head.next_3012.next_3012.next_3012.prev_3012 =
head.next_3012.next_3012;
        head.next_3012.next_3012.next_3012.next_3012 = new
NodeDLL_2511533012(5);
        head.next_3012.next_3012.next_3012.next_3012.prev_3012
= head.next_3012.next_3012.next_3012;

        System.out.print("DLL Awal: ");
        printList_3012(head);

        System.out.print("Setelah head dihapus: ");
        head = delHead_3012(head);
        printList_3012(head);

        System.out.print("Setelah node terakhir dihapus: ");

```

```

        head = delLast_3012(head);
        printList_3012(head);

        System.out.print("Menghapus node ke 2: ");
        head = delPos_3012(head, 2);

        printList_3012(head);
    }
}

```

Output :

```

<terminated> HapusDLL_2511533012 [Java Application] C:\Program
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
Menghapus node ke 2: 2 4

```

Analisis :

Program ini digunakan untuk melakukan operasi **penghapusan node** pada Doubly Linked List.

Operasi yang tersedia meliputi:

- *delHead* untuk menghapus node pertama
- *delLast* untuk menghapus node terakhir
- *delPos* untuk menghapus node pada posisi tertentu

Program juga memperbarui hubungan antar node setelah proses penghapusan dilakukan.

Kesimpulan: Program ini menjelaskan proses penghapusan node pada Doubly Linked List.

BAB III

KESIMPULAN

3.1 Ringkasan

Pada praktikum pekan ke-6, mahasiswa mempelajari implementasi Doubly Linked List menggunakan Java. Praktikum mencakup pembuatan node, penambahan data, penghapusan data, dan traversal dua arah. Mahasiswa juga memahami penggunaan pointer next dan prev dalam menghubungkan antar node.

3.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Doubly Linked List merupakan struktur data yang memungkinkan penelusuran data dilakukan dari dua arah. Dengan adanya pointer next dan prev, proses manipulasi data menjadi lebih fleksibel dibandingkan Single Linked List. Praktikum ini membantu mahasiswa memahami konsep dasar linked list dua arah sebagai dasar untuk mempelajari struktur data yang lebih kompleks.

DAFTAR PUSTAKA

- [1] Oracle, "The Java Tutorials," 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>.
- [2] H. M. Deitel dan P. J. Deitel, Java: How to Program, 10th ed. Boston: Pearson, 2015

Tugas Pratikum

1. Lagu_2511533012.java

```
package pekan6_2511533012;

public class Lagu_2511533012 {

    private String judul_3012;
    private String penyanyi_3012;

    Lagu_2511533012 next_3012;
    Lagu_2511533012 prev_3012;

    public Lagu_2511533012(String judul_3012, String penyanyi_3012) {

        this.judul_3012 = judul_3012;
        this.penyanyi_3012 = penyanyi_3012;

        this.next_3012 = null;
        this.prev_3012 = null;
    }

    public String getJudul_3012() {
        return judul_3012;
    }

    public String getPenyanyi_3012() {
        return penyanyi_3012;
    }

    public void setJudul_3012(String judul_3012) {
        this.judul_3012 = judul_3012;
    }

    public void setPenyanyi_3012(String penyanyi_3012) {
        this.penyanyi_3012 = penyanyi_3012;
    }
}
```

2. Musik_2511533012

```
package pekan6_2511533012;
import java.util.Scanner;

public class Musik_2511533012 {

    Lagu_2511533012 head_3012;
    Lagu_2511533012 tail_3012;

    public void tambahLagu_3012(String judul_3012, String penyanyi_3012) {
```

```

        Lagu_2511533012 laguBaru_3012 = new Lagu_2511533012(judul_3012,
        penyanyi_3012);

        if (head_3012 == null) {

            head_3012 = laguBaru_3012;
            tail_3012 = laguBaru_3012;

        } else {

            tail_3012.next_3012 = laguBaru_3012;
            laguBaru_3012.prev_3012 = tail_3012;
            tail_3012 = laguBaru_3012;

        }

        System.out.println("Lagu berhasil ditambahkan!");
    }

    public void hapusLaguAwal_3012() {

        if (head_3012 == null) {

            System.out.println("Playlist kosong!");
            return;

        }

        System.out.println("Lagu " + head_3012.getJudul_3012() + "
        berhasil dihapus.");

        if (head_3012 == tail_3012) {

            head_3012 = null;
            tail_3012 = null;

        } else {

            head_3012 = head_3012.next_3012;
            head_3012.prev_3012 = null;

        }

    }

    public void tampilMaju_3012() {

        if (head_3012 == null) {

            System.out.println("Playlist kosong!");
            return;

        }

        Lagu_2511533012 bantu_3012 = head_3012;

        System.out.println("\n=== Playlist Maju ===");
    }

```

```

        while (bantu_3012 != null) {
            System.out.println(bantu_3012.getJudul_3012() + " - " +
bantu_3012.getPenyanyi_3012()
            );
            bantu_3012 = bantu_3012.next_3012;
        }
    }

    public void tampilMundur_3012() {
        if (tail_3012 == null) {
            System.out.println("Playlist kosong!");
            return;
        }

        Lagu_2511533012 bantu_3012 = tail_3012;

        System.out.println("\n=== Playlist Mundur ===");

        while (bantu_3012 != null) {
            System.out.println(
                bantu_3012.getJudul_3012() + " - " +
bantu_3012.getPenyanyi_3012()
            );
            bantu_3012 = bantu_3012.prev_3012;
        }
    }

    public void cariLagu_3012(String judulCari_3012) {
        if (head_3012 == null) {
            System.out.println("Playlist kosong!");
            return;
        }

        Lagu_2511533012 bantu_3012 = head_3012;

        boolean ketemu_3012 = false;

        while (bantu_3012 != null) {
            if
(bantu_3012.getJudul_3012().equalsIgnoreCase(judulCari_3012)) {
                System.out.println("\nLagu ditemukan!");
            }
        }
    }

```

```

        System.out.println("Judul : " +
bantu_3012.getJudul_3012());

        System.out.println("Penyanyi : " +
bantu_3012.getPenyanyi_3012());

        ketemu_3012 = true;

        break;
    }

    bantu_3012 = bantu_3012.next_3012;
}

if (!ketemu_3012) {
    System.out.println("Lagu tidak ditemukan!");
}
}

// Main Program
public static void main(String[] args) {

    Scanner input_3012 = new Scanner(System.in);

    Musik_2511533012 musik_3012 = new Musik_2511533012();

    int pilihan_3012;

    do {
        System.out.println("\n=== Playlist Musik NIM: 2511533012
===");

        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");

        System.out.print("Pilihan : ");

        pilihan_3012 = input_3012.nextInt();
        input_3012.nextLine();

        switch (pilihan_3012) {

            case 1:

                System.out.print("Judul Lagu : ");
                String judul_3012 = input_3012.nextLine();

```

```

        System.out.print("Penyanyi : ");
        String penyanyi_3012 = input_3012.nextLine();

        musik_3012.tambahLagu_3012(
            judul_3012,
            penyanyi_3012
        );

        break;

    case 2:

        musik_3012.hapusLaguAwal_3012();

        break;

    case 3:

        musik_3012.tampilMaju_3012();

        break;

    case 4:

        musik_3012.tampilMundur_3012();

        break;

    case 5:

        System.out.print("Masukkan judul lagu : ");

        String cari_3012 = input_3012.nextLine();

        musik_3012.cariLagu_3012(cari_3012);

        break;

    case 6:

        System.out.println("Program selesai.");

        break;

    default:

        System.out.println("Pilihan tidak tersedia!");

    }

} while (pilihan_3012 != 6);

input_3012.close();
}

```

```
}
```

3. Outpun Syntax program

```
2.1  
=== Playlist Musik NIM: 2511533012 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan :
```

```
=== Playlist Musik NIM: 2511533012 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan : 1  
Judul Lagu : loneliness  
Penyanyi : zutomayo  
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511533012 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan : 1  
Judul Lagu : school of hard knock  
Penyanyi : high therapy, tuantigabelas  
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511533012 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan : 5  
Masukkan judul lagu : loneliness  
  
Lagu ditemukan!  
Judul : loneliness  
Penyanyi : zutomayo
```

```

=== Playlist Musik NIM: 2511533012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 3

=== Playlist Maju ===
loneliness - zutomayo
school of hard knock - high therapy, tuantigabelas

=== Playlist Musik NIM: 2511533012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 4

=== Playlist Mundur ===
school of hard knock - high therapy, tuantigabelas
loneliness - zutomayo

```

```

=== Playlist Musik NIM: 2511533012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu loneliness berhasil dihapus.

```

```

=== Playlist Musik NIM: 2511533012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 6
Program selesai.

```

Penjelasan:

Program ini dibuat untuk mengelola playlist lagu menggunakan struktur data **Double Linked List**. Pada struktur ini, setiap data lagu saling terhubung menggunakan dua pointer, yaitu pointer ke data berikutnya dan pointer ke data sebelumnya. Dengan adanya dua hubungan tersebut, playlist dapat ditampilkan dari awal ke akhir maupun dari akhir ke awal.

Program terdiri dari dua class, yaitu Lagu_2511533012 dan Musik_2511533012. Class Lagu_2511533012 digunakan sebagai node untuk menyimpan data lagu berupa judul dan penyanyi. Selain itu, class ini juga memiliki pointer next dan prev yang digunakan untuk menghubungkan node satu dengan node lainnya dalam Double Linked List.

Sedangkan class Musik_2511533012 digunakan untuk mengatur seluruh proses pada playlist musik. Pada class ini terdapat pointer head yang menunjuk lagu pertama dan tail yang menunjuk lagu terakhir. Dengan adanya head dan tail, proses penambahan, penghapusan, dan penampilan data dapat dilakukan dengan lebih mudah.

Program memiliki beberapa fitur utama. Fitur pertama adalah menambahkan lagu ke akhir playlist. Saat lagu baru ditambahkan, node baru akan dihubungkan dengan node terakhir sebelumnya sehingga hubungan antar data tetap tersusun dengan benar. Jika playlist masih kosong, maka lagu pertama akan menjadi head sekaligus tail.

Fitur berikutnya adalah menghapus lagu pertama pada playlist. Ketika lagu pertama dihapus, posisi head akan berpindah ke lagu berikutnya. Jika hanya terdapat satu lagu, maka setelah dihapus playlist akan kembali kosong. Program juga memiliki fitur menampilkan playlist secara maju dan mundur. Tampilan maju dilakukan dengan membaca data mulai dari head menuju tail menggunakan pointer next. Sedangkan tampilan mundur dilakukan mulai dari tail menuju head menggunakan pointer prev. Fitur ini menunjukkan fungsi utama dari Double Linked List karena data dapat diakses dari dua arah.

Selain itu, terdapat fitur pencarian lagu berdasarkan judul. Program akan menelusuri setiap node pada playlist hingga menemukan judul lagu yang sesuai. Jika lagu ditemukan, maka data lagu akan ditampilkan, sedangkan jika tidak ditemukan program akan menampilkan pesan bahwa lagu tidak ada di playlist.

Pada bagian utama program terdapat menu interaktif yang memungkinkan pengguna memilih operasi yang ingin dijalankan, seperti

menambah lagu, menghapus lagu, melihat playlist, mencari lagu, dan keluar dari program. Program akan terus berjalan hingga pengguna memilih menu keluar.

Secara keseluruhan, program ini menerapkan konsep Double Linked List untuk mengelola data playlist musik secara dinamis. Struktur data ini memudahkan proses penambahan, penghapusan, pencarian, serta penampilan data dari dua arah dengan lebih efisien.

.